



.....
عنوان : طراحی و توسعه تعاملی مدیریت و پالایش و تحلیل داده های موسیقی با استفاده از زبان برنامه نویسی پایتون .

- 1 . ابتدا ایجاد ریپازیتوری تو اکانت گیت هابم .
- 2 . ایجاد فایل ها و فولدر های لازم داخل فولدر ریپازیتوری .
- 3 . نوشتن کلاس اهنگ (سانگ) داخل فایل جدای پایتون برای تبدیل هر یک از اهنگ های فایل به یک شی پایتون .
- 4 . اضافه کردن شروط معتبر بودن سه تا از متغیر ها با استفاده از پراپرتی و ستر و گتر در شی گرایی پایتون .
- استفاده از پراپرتی تو این جا به دو دلیل : عدم استفاده از پارانتز موقع استفاده و اعتبارسنجی برای جلوگیری از مقادیر نامعتبر.
- 5 . نوشتن کلاس دیتا لودر داخل فایل دیتا لودر برای خواندن دیتا از فایل موجود و اضافه کردن دیتا به اخر ان .
- 6 . اضافه کردن متدی برای خواندن فایل اهنگ ها و اضافه کردن ها به یک لیست در ان کلاس .
- 7 . اضافه کردن متد دیگری به کلاس دیتا لودر برای اضافه کردن اهنگ (ترک) جدید به انتهای فایل اهنگ ها . باید توجه کنیم که اهنگی که به این تابع میدیم تا اضافه کنه به لیست اهنگ هامون از جنس کلاس سانگی که قبلا درست کردیم باشه و اول مشخصات اهنگ رو بگیریم و سپس یه شی اهنگ بسازیم بعد این شی رو پاس بدیم به تابع مدنظرمون .
- 8 . میتوانستم خواندن همه 100.000 اهنگ از فایل را با استفاده از جنریتور هم انجام بدم ولی از انجا که در مازول های بعدی و مرحله های بعدی به همه اهنگ ها در یک جا نیاز خواهیم داشت برای محاسبه میانگین و میانه و دیگر ابزار لازم و ... استفاده از لیست معمولی برای ذخیره انها هم از لحاظ حافظه و هم از لحاظ زمان به صرفه تر بود .
- ***این رو هم اضافه کنم که خواندن و نوشتن تو این مازول رو همش رو با استفاده از امکانات و توابع کتابخانه سی اس وی انجام دادم و هندل کردم .
- ***یه چیزی که بعدا متوجه شدم باید به کلاس دیتا لودر اضافه کنم : دو متد یکی برای ذخیره همه تغییراتی که تا به جایی که خودمون بخوایم روی داده هامون انجام دادیم که اضافه کردیم که میاد یه فایل سی اس وی جدید باز میکنه و لیستی که از اهنگ هامون داریم رو با استفاده از کتابخانه سی اس وی توی فایل جدیدمون مینویسه . دومین متد هم برای ریست کردن دیتا هست . از اول پروژه برام سوال بود که مثلا یک کاربر داده ها رو یجوری مرتب میکنه بعد نمودار ایناش رو چاپ میکنه بعد میخواد دوباره با یه روش دیگه ای از اول داده ها رو مرتب کنه !!! این متد برای حل این مسئله و خیلی سادست . میاد لیستی که داریم رو خالی میکنه بعد دوباره اهنگ های اولیه تو فایل دیتاست مون رو میریزه تو لیست ...
- 9 . شروع نوشتن مازول دیتا کلینر و در ابتدا یه فرم درختی براش ساختم تا بعدا یکی یکی شاخه ها رو تکمیل کنم .
- 10 . اولین کلاس تو این مازول هیچ قابلیت نداره و فقط نمادین و انتزاعیه .
- 11 . دومین کلاس تو این مازول برای پرکردن داده های گمشده با استفاده از میانگین هستش که اول اومدم روی لیستی که قراره به تنها متد این کلاس پاس بدیم حرکت کردم و اون ویژگی از اهنگ ها رو که میخوایم روش تحقیقات انجام بدیم رو هم پاس میدم به این متد بعد با استفاده از دو تابع جذاب و جالب آماده پایتون برای شی گرایی به مقادیر این ویژگی هر اهنگ دسترسی پیدا میکنیم و اگر

گمشده بود یعنی خالی یا عدد صفر یا رشته صفر بود در نظر نمیگیریمش ولی بقیه رو به یه لیست جدید اضافه میکنیم بعد با استفاده از کتابخونه نامپای پایتون میانگین لیست جدید رو میگیریم بعد دوباره روی عناصر اولیه حرکت میکنیم و اگر همچنان مقدار اون ویژگی گمشده به حساب میومد اون میانگین بدست اومده رو جایگزین مقدارش میکنیم ...

12. همه کارهای بالا روبرای میانه هم عینا انجام میدیم ولی با این تفاوت که از تابع میانه نامپای بجای تابع میانگین نامپای استفاده میکنیم و سومین کلاس هم ساختش تو این ماژول تموم میشه ...

13. و اما شاید بتونم بگم چالش اول تو این پروژه این محصل: متدی تو بخش جایگذاری داده های گمشده با استفاده از ابزار و امکانات کتابخانه اسکی لرن و ماشین لرنینگ. خیلی سردر گم بودم که از کجا باید شروع کنم و چجوری شروع کنم اصلا چی هست ... که بعد از سرچ و تحقیق شروع به نوشتنش کردم هر چند کوتاه بود و بیشتر از چند خط نشد فعلا. تو شروع اومدم به لیست 9 تایی از اسم ویژگی هایی که عددی هستن ساختن بعد به ماتریس خالی تعریف کردم که قرار بود همه مقادیر اعم از عدد یا گمشده ها (رشته خالی یا عدد صفر یا رشته صفر) که به متغیر پوچ تو نامپای تبدیلشون کردم تا ماشین بفهمه این متغیر گمشده و باید پرش کنه) بعد اومدم با دو تا فور تو در تو روی لیست اهنکام با ویژگی هاشون حرکت کردم و کاری که بالا گفتم رو انجام دادم اینطوری که هر سطر تو ماتریس نهاییم بشه برا یه اهنک چون که تو ماشین هر سطر باید یک شی باشه و هر ستون بشه یه ویژگی از اون اهنک که اگه گمشده بود بجاش پوچ میزارم. خب بعدش اومدم ماشینمو ساختم و تو خط بعدی که مهمترین خط این متد هست با استفاده از متد فیت ترنسفرم به ماشینم یاد دادم که با استفاده از پنج تا از نزدیکترین همسایه ها داده های گمشده رو جایگذاری کنه. و بعد تو مرحله اخر اومدم به ماتریس با استفاده ماشینم ساختم که ماتریس اولیه رو بهش ورودی دادم و تو ماتریس ساخته شده نهایی هیچ داده و متغیری پوچ یا همون گمشده نیست و همه مقدار دارن. و در نهایت دوباره با استفاده از دو تا حلقه فور تو در تو روی ماتریس جدید حرکت کردم. مقادیر جدید رو توی لیست اهنک هام که به این تابع ورودی داده بودم اپدیت کردم.

و اما بخش جالب تو این مرحله نقض چند ریختی تو شی گرایی پایتون بود چون که چه تو کلاس پایه و چه تو کلاس های قبلی به تنها متد این کلاس ها توی ورودی اسم ویژگی مدنظرمون رو میدادیم ولی تو اینجا که داریم با ماشین کار میکنیم نمیشد اسم ویژگی ورودی داد چون باید روی داده های گمشده با ماشین عملیات انجام بدیم و پنج تا از نزدیکترین همسایه های یه اهنک دارای داده گمشده تو برخی ویژگی ها رو پیدا کنیم و ماشین اون داده گمشده رو بدست بیاره پس به همین خاطر برای تشخیص همسایه های نزدیک باید همه ویژگی های عددی رو در نظر بگیریم.

این ارور رو میداد:

```
ModuleNotFoundError: No module named 'sklearn'
PS C:\Users\SAYAR-ELECTRONICS\Desktop\spotify-studio> python -u "c:\Users\SAYAR-ELECTRONICS\Desktop\spotify-studio\src\data_cleaner.py"
Traceback (most recent call last):
  File "c:\Users\SAYAR-ELECTRONICS\Desktop\spotify-studio\src\data_cleaner.py", line 72, in <module>
    k.impute(["s", "q"])
TypeError: KNNImputer.impute() missing 1 required positional argument: 'feature_name'
PS C:\Users\SAYAR-ELECTRONICS\Desktop\spotify-studio>
```

که از این راه حل استفاده کردم و حل شد:

```
class KNNImputer(BaseImputer):
    def impute(self, track_list, feature_name = None):
```

14. خب حالا بخش مدیریت داده های پرت رو شروع میکنیم. کلاس والد یا همون کلاس پایه تو اینجا هم مٹ بخش قبلی فقط یه چیز نمادین و انتزاعی و ظاهریه و فقط دارای یک متد با بدنه خالی هستش.

15. اولین زیر کلاس تو این بخش مربوط به مدیریت داده های پرت با استفاده از دامنه میان چارکی هستش. بخش خوندن و اپدیت کردن لیست اهنک ها دقیقا شبیه کلاس ها و بخش های قبلی هست. به اینصورت که ابتدا به لیست خالی میسازیم بعد روی لیست اهنک ها حرکت میکنیم و اون ویژگی که به تنها متد کلاسمون پاس دادیم و میخونیم و به اون لیست خالی اضافه میکنیم. بعدا با استفاده از توابع آماده پکیج نامپای تو پایتون ابتدا چارک اول لیست و سپس چارک سوم لیست را بدست می اوریم. در متغیر های عددی ذخیره میکنیم سپس با تفاضل این دو مقدار از هم _ دامنه میان چارکی را بدست می اوریم. سپس عدد 1.5 را در دامنه ضرب میکنیم و ان را از چارک اول کم میکنیم تا شروع دامنه یا همان حداقل مقدار مجاز بدست میاد. برای حداکثر مقدار مجاز هم چارک

سوم را علاوه ضرب دامنه در عدد 1.5 و دامنه . بدست می آوریم . سپس دوباره با حلقه ها روی لیست اهنگ ها حرکت میکنیم و اگر ویژگی مدنظر تو به اهنگ بیشتر از حداکثر مقدار مجاز بود داده پرت به حساب آمده و آن را برابر با حداکثر مقدار مجاز قرار میدیم و یا اگر کمتر از حداقل مقدار مجاز بود داده پرت به حساب آمده و آن را برابر با حداقل مقدار مجاز قرار میدیم .

16. **چالش دوم:** و اما به بخش جذاب دیگه.. زیر کلاس دوم توبخش مدیریت داده های پرت که با استفاده از چیزی بنام **نمره استاندارد** انجام میشد نمیدونم اسمشو بذارم به **چالش** تو این پروژه یا نه؟! چون هیچ چیزی از این روش نمیدونستم و بعد خوندن و سرچ و تحقیق در موردش چیز زیاد سخت و طاقت فرسایی به نظر نیومد بلکه جالب بود بنظرم .. اول به خلاصه بگم که این روش چیه اصلا؟؟

معرفی خلاصه وار روش نمره استاندارد : این روش یک ابزار اماری هوشمند برای سنجش میزان عجیب بودن بودن داده هاست در این روش ابتدا میانگین و انحراف معیار را حساب میکنیم سپس طبق یک قانون جهانی امار مرز های بالا و پایین مجاز را روی فاصله سه برابری از انحراف معیار تنظیم میکند (یا به عبارتی برای هر داده نمره استاندارد آن را که از فرمول (داده - میانگین) تقسیم بر انحراف معیار بدست می آید را حساب میکنیم و اگر این عدد بین -3 و +3 بود یعنی داده پرت نیست در غیر این صورت داده پرت به حساب می آید) و داده های پرت را به صورت خودکار به مرزهای قانونی تعیین شده کات و اصلاح میکند تا رفتار عجیب آن کل محاسبات و تحلیل های بعدی دیتابیس اسپاتیفای را خراب نکند ... من هم در این پروژه برای بهینه تر بودن و صرفه جویی در زمان و حافظه از محاسبه نمره استاندارد برای تک تک داده ها پرهیز کردم و با طرفین وسطین کردن فرمول ذکر شده بازه مجاز را با استفاده از اعداد -3 و +3 بدست آوردم .

خب حالا بعد از معرفی این روش به ادامه کارمون بپردازیم ... بخش خوندن دیتا ها و داده ها و اپدیت کردن آنها در لیست اولیه اهنگ ها دقیقاً مثل زیر کلاس قبلی است . بقیه بخش ها هم مثل حالت قبل است فقط با این تفاوت که در کلاس حداقل و حداکثر مقدار مجاز را با استفاده از دامنه میان چارکی تعیین میکردیم ولی در این کلاس حداقل و حداکثر مقدار مجاز را با استفاده از فرمول نمره استاندارد که در بالا بهش اشاره کردم (با استفاده از اعداد -3 و +3) تعیین میکنیم .

17 . خب نوبتی هم باشد نوبت رفتن به سراغ تکمیل فایل دیتا آنالیز بود . تو بخش ما قراره دیتاست مون رو آنالیز کنیم و چیز های اماری راجع بهش بدست آوریم و چند متد جدای دلخواه . تو این فایل من فقط یک کلاس بنام دیتاآنالیز درست کردم که قراره متد های متعددی داشته باشه ... تنها متد جادویی تو این کلاس فقط کارش اینه که لیست اهنگ ها رو بگیره (اهنگ هایی که هر کدوم یک شی هستند) تا از تکرار این بخش تو متد های بعدی جلوگیری بشه .

18 . هفت متد اصلی اول تو این کلاس راجب محاسبات اماری داده ها بود . اول میخواستم تو این تحلیل هر کدوم از اونارو جدا جدا بنویسم بعد چون که خیلی شبیه هم بودند و فقط در دو یا چند فرمول و اسم تفاوت داشتند تصمیم گرفتم تصمیم همه اناه را در یک بخش اینجا بهتون توضیح بدم . اینجوریه که من هفت تا ویژگی مهم اماری داده ها رو بدست میارم : (میانگین . میانه . واریانس . انحراف معیار . مد . ماکسیمم . مینیمم) . اول میایم تو ورودی تابع اسم ویژگی از اهنگ ها که میخوایم اینارو روشون حساب کنیم رو میگیریم بعد مثل همیشه روی لیست اهنگ ها حرکت میکنیم و اعداد مربوط به اون ویژگی ورودی داخل به لیست میریزیم بعد لیست بوجود آمده رو به اری نامپای تبدیل میکنیم و بعد برای محاسبه متغیر های اماری از توابع آماده پکیج نامپای استفاده کردم .

19 . متد هشتم و نهم تو این فایل دو متد دلخواه برای فیلتر کردن اهنگ ها بودن اینطوری که یکیش فیلتر کردن توسط اسم هنرمند بود که اهنگ ها رو بر اساس اسم هنرمندی که کاربر ورودی میده هست و خروجیش به لیست از اهنگ ها با هنرمند یکسان هست .

دومیش فیلتر کردن توسط ژانر بود که کاربر ژانر مورد علاقتو ورودی میداد و خروجی لیستی از همه اهنگ ها با اون ژانر بود *** میتوان تعداد اهنگ های خروجی داخل لیست خروجی رو هم با به ورودی دیگه به راحتی هندل کرد .

20 . متد بعدی راجب پیدا کردن برترین اهنگ ها از لحاظ ویژگی که کاربر خودش ورودی میده به تعداد مدنظر کاربر . اینطوریه که تابع مون میاد لیست اهنگ هامون با استفاده از تابع آماده سورت نسبت به ویژگی مد نظر کاربر با استفاده از تابع ناشناس لامبدا از بزرگ به کوچک مرتب میکنه و میریزه داخل به لیست جدید بعد تعداد موردنظر کاربر رو از لیست جدید خروجی میده .

21 . متد اخر تو این کلاس برای پیشنهاد اهنگ به کاربر با اهنگ ورودی مورد علاقه کاربر و ویژگی مدنظر کاربر و همچنین تعدا اهنگی که میخواد دریافت کنه هستش . اینطوریه که اول میاد اون ویژگی اهنگ کاربر رو حساب میکنه و ذخیره میکنه بعد دوباره میاد لیست اهنگ ها رو مرتب میکنه ولی ایندفعه با تابع ناشناسی که کلیدش تفاوت مقدار اون ویژگی تو اهنگ با وقدار اون ویژگی تو اهنگ کاربر هستش . بعد تعداد اهنگ مدنظر کاربر رو به کاربر خروجی میده . تو اینجا هم اهنگی که پاس میدیم به این تابع باید از جنس شی کلاس سانگ باشه .

یچیز جالب و شاید سردرگم کننده تو این بخش این بود که مثلا ما تو اکثر متد هامون یه لیست از اهنگ ها داشتیم و از اون خیلی استفاده میکردیم و در نهایت هم دوباره یه لیست از اهنگ ها برمیگردوندیم . چیزی که شاید مثل من برای خیلیا سوال باشه و فکرشون رو درگیر کنه اینه که تو این پروژه از اول هر اهنگ رو یک شی در نظر گرفتیم و با قوانین وامکانات و توابع شی گرایی باهاش رفتار کردیم و کار کردیم و مورد قابل توجه اینه که چطور قراره با یه لیستی از اهنگ ها که خروجی میدی در بخش های بعدی کار کنیم؟! تو جواب باید بگم که واقعا هیچ فرقی با شی معمولی ندارن و موقع کار با اهنگ ها تو لیست خروجی هم دقیقا مثل قبل رفتار میکنیم و طبق قوانین اصلی شی گرایی میتوان به تمام اطلاعات و متد های اهنگ ها دسترسی پیدا کرد .

22 . خب الان دیگه وقتشه بریم سراغ تکمیل فایل جذاب دیتاویژولایزر که قراره توش رسم نمودار ها رو تکمیل کنیم ... فقط یه کلاس با متد های متفاوت داخل این فایل خواهیم داشت . داخل متد این ایت این کلاس فقط داریم لیست اهنگ رو ورودی میدیم که تو بخش های بعدی که قراره لازم داشته باشیم دیگه تکرار نکنیمش ... و اما متد های بعدی که قراره جلو تر راجب تک تک شون بحث کنیم درباره رسم انواع نمودار هاس : نمودار باکس پلات و اسکتر پلات و ماتریس همبستگی .

23 . اولین متد راجب رسم نمودار باکس پلات یا همون نمودار جعبه ای هستش که قراره بیار قبل تمیزکاری و بیار بعد تمیزکاری نمایشش بدیم تو خروجی . این تابع مون دو تا ورودی میگیره یکیش ویژگی ای از اهنگ هاس که میخوایم نمودار رو دربارش رسم کنیم و یکیش هم یه رشتس که مشخص میکنه چجوری داده ها رو تمیزکاری کردیم و اینکه میگه نمودار نمایش داده شده قبل تمیز کاریه یا بعدش . اینجوریه که روی لیست اهنگ ها حرکت میکنیم و اون ویژگی ورودی اهنگ ها رو داخل یه لیست جدید ذخیره میکنیم بعد اون لیست رو به نمودارمون میدیم . وقتی یه نمودار رو ایجاد میکنیم در واقع یه شی ایجاد میشه . بعدش اومدیم تنظیمات مربوط به رنگ و ضخامت خطوط و غیره رو تو نمودارمون با استفاده امکانات پکیج مت پلات لیب هندل کردیم که مثلا قبل تمیز کاری قرمز باشه رنگ داخل جعبه مون بعدش سبز باشه و رنگ خطوط و ... رو تنظیم کردیم . **اون حلقه ها** رو برای اینکه ممکنه چند تا نمودار جعبه ای همزمان چاپ بشه گذاشتیم تا همش رو بتونیم مدیریت کنیم و از انجایی که پایتون نمیتونه به رنگ یه لیست رو تغییر بده و فقط میتونه به عنصر داخلش اگر چه بدونه باشه دسترسی پیدا کنه و رنگ اونو تغییر بده پس باید یا با حلقه به عناصر لیست دسترسی پیدا کنیم یا با استفاده از ایندکس که ما حلقه رو انتخاب کردیم.

24 . دومین متد هم راجب نمودار پراکندگی بود که قراره بیار تو خروجی اونم بعد تمیز کاری چاپ بشه . این تابع دو تا اسم ورودی که قراره محور های افقی و عمودی رو تشکیل بدن رو میگیره . بعد میایم روی لیست اهنگ ها حرکت میکنیم و ویژگی های مربوط به اون دو تا رو داخل لیست های جدا ذخیره میکنیم سپس داخل یه لیست هم اسم ژانر ها رو ذخیره میکنیم بعد از اون لیست با امکانات کتابخانه کانتیر ده تا از تاپ ترین ژانر ها رو تو یه لیست جدا ذخیره میکنیم و بعد یه لیست ایندکسی از روی اون لیست میسازیم . حالا وقت ساختن نموداره که قراره دو تا لیست ویژگی هرو بهش ورودی بدیم و الفا که مقدار شفافیت رو نشون میده رو هفت دهم میذاریم . بعد لیست ایندکس ها را در متغیر (سی) ورودی بهش میدیم و پایتون با این کار میفهمه نباید همه نقاط را یک رنگ کند . و داخل متغیر (سی مپ) ورودی بهش یه اسم یه استاندارد رو به صورت رشته مینویسیم . نمودار رو میسازیم. بعد با استفاده از متد لجند تو کتابخانه مت پلات لیب اون کادر راهنما رو کنار نمودارمون میسازیم که تو ورودی هندلس رو لجند المنت نمودارمون میسازیم که خودکار تشخیص میده چه رنگ روی صفحه رسم شده اند و با گذاشتن {0} میگوییم فقط ان دایره های رنگی را به عنوان هندل انتخاب کن نه اسم رنگ ها را. برای هر رنگ یک دایره کوچک رنگی خلق میکند . بعد لیست ژانر های منحصر به فرد رو بهش میدیم و به متغیر تایتل با نام رشته ای ژانر ها هم بهش ورودی میدیم . هندل و لیبیل ها مثل یک چرخ دنده و مکمل به هم متصل میشوند . بعد در مرحله اخر هم مراحل رسم نمودار و ... رو ادامه میدیم . **متد جی سی آ از کتابخونه مت پلات لیب هم نمودار فعلی رو صفحه رو برمیگردونه .**

***** فقط باید توجه بشه که این بخش جدول راهنما که ژانر ها رو نشون میده مستقل از اون دو تا ویژگی ای هست که بهش ورودی میدیم و به عنوان یه بعد سوم عمل میکنه برامون و برای اینکه ارور نده و درست نمایش داده بشه فقط ده تا ژانر رو از بین اون همه ژانر برای برای نمودار پراکندگی انتخاب کردیم .**

**** مورد بعدی که برای شخص من جالب بود اینه که بر خلاف رسم نمودار های ساده و معمولی که یاد گرفته بودیم اینجا باید موقع ایجاد نمودار اونو به عنوان یه شی تو متغیری ذخیره کنیم چون که در خط های بعدی تابع مون لازمشون داریم . در جدول راهنما تو نمودار پراکندگی و در جعبه و داخل جعبه و ... تو نمودار جعبه ای .**

```
box = plt.boxplot(t2 , patch_artist=True)
```

اون متغیر بولین هم برای این هست که بتونیم داخل نمودار جعبه ای مون رو رنگ امیزی کنیم .

25. **چالش سوم :** و اما بخش آخر تو این فایل یا همون ماتریس همبستگی . که من هیچ درک و شهودی ازش نداشتم و اصلا نمیدونستم چیه . بعد از تحقیق و سرچ فهمیدم که میشه با استفاده از کتابخانه سیپورن به راحتی پیاده سازیش کرد ولی از اونجا که من با این کتابخانه اصلا آشنا نبودم ترجیح دادم خودم فرمول هایش را بنویسم و با استفاده از توابع و امکانات خود پایتون و پکیج مت پلات لیب پیاده سازیش کنم . اول بیاین بهتون یه توضیح کوچیک بدم که اصلا این روش هی هستش !!

توضیح خلاصه وار روش پیاده سازی ماتریس همبستگی : یک جدول شطرنجی و متقاطع است که نشان میده و ویژگی های مختلف یک مجموعه داده (تو این پروژه اهنگ ها مثلا ویژگی های صوتی و انرژی و محبوبیت و رقص انگیزی و ...) چقدر با هم رابطه دارند و تغییرات یکی چطور روی دیگری اثر میگذارد . اعداد داخل این ماتریس همیشه بین -1 تا +1 هستند :

+1 : همبستگی مثبت کامل . هر چه ویژگی اول بیشتر شود ویژگی دوم هم به همان نسبت بیشتر میشود مثل انرژی و ولوم

-1 : همبستگی منفی کامل . هر چه ویژگی اول بیشتر شود ویژگی دوم کمتر میشود مثل اکوستیک بودن و انرژی .

0 : عدم همبستگی . هیچ رابطی بین دو ویژگی وجود ندارد و به هم ربط ندارند .

فرمول ریاضی برای محاسبه همبستگی بین دو ویژگی :

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

خب حالا ادامه بدیم . اول میایم به تابع مون لیستی از ویژگی ها رو که میخوایم دو به دو همبستگی بین اونا رو بررسی و محاسبه کنیم ورودی میدیم . بعد یه دیکشنری میسازیم که کلیداش اسمای این ویژگی ها هستن و مقدار هاش لیستی از مقادیر ویژگی اهنگ ها تو این ویژگی . بعد میایم یه ماتریس مربع صفر با ابعاد تعداد ویژگی هایی که بهش دادیم درست میکنیم . بعد میایم با دو تا حلقه فور تو در تو رنج تعداد ویژگی ها حرکت میکنیم و اونا رو به ترتیب دو به دو جدا میکنیم و تو هر مرحله میزان همبستگی رو با استفاده از فرمول بالا محاسبه میکنیم و اگر مخرج صفر نبود (با یه شرط چک میکنیم) تو ماتریس جای مناسب خودش جایگذاری میکنیم و اگر مخرج صفر بود همون عدد صفر رو جایگذاری میکنیم . خب الان بخش محاسبه تموم شد و موند فقط بخش رسم جدول برای رسم جدول ابتدا صفحه رو پاکسازی میکنیم بعدش یه شکل جدید میسازیم بعد تابع ایمشو تو شکل جدیدمون میاد ماتریس دو بعدی مون رو به یک جدول رنگی تبدیل میکنه . پالت رنگی استاندارد بی دلبیو آر همان طیف رنگی است که خانه های جدول را رنگ میکند برای اعداد منفی رنگ ابی تیره و برای مثبت رنگ قرمز تیره و برای صفر رنگ سفید اختصاص میده و داخل همین متد بهش میگیم کف رنگ ابی تیره روی عدد -1 و سقف رنگ قرمز تیره روی عدد +1 قفل شود . تابع کالربار ان نوار عمودی مدرج را در سمت راست نمودار میکشد تا راهنمای سنجش رنگ ها باشد بعد تو خط بعدی هم یک برچسب لیبل عمودی به ان میچسباند تا مشخص شود این نوار مربوط به ضریب همبستگی است . بعدا مت پلات لیب به صورت پیش فرض سطر ها و ستون ها را از عدد صفر شماری گذاری میکند ولی با توابع مناسب تیکت های مناسب هر محور را ست میکنیم با نام ویژگی ها بعد برای محور ایکس یک زاویه چرخش هم میسازیم تا اگر نام ویژگی ها طولانی بودند در هم فرو نروند و کامل نمایش داده شوند . بعد تو مرحله آخر با دو تا حلقه فور تو در تو روی تک تک اعضای ماتریسمون حرکت میکنیم . بخاطر کم رنگ یا پررنگ بودن رنگ هر خانه میایم رنگ نوشته با سفید و سیاه تنظیم میکنیم . بعد عدد رو تا دو رقم اعشار بصورت بولد شده تو مرکز هر خانه تو جدول مون چاپ میکنیم . و در نهایت هم با استفاده از متد تایت لی اوت جدول رو به صورت خودکار منظم و مرتب میکنیم و فاصله اینارو هم تنظیم میکنیم و تمام .

بایه پارامتر مخصوص بهش میگیم تراز عمودی (متن نسبت به نقطه ای که قر گرفته چطور تراز شود) پایین متن روی نقطه قفل شود.

اضافه کردن یه بخش امتیازی به کلاس نمودار هامون : (نمودار واژه ابر) . برای نوشتن باید کتابخانه مربوط بهش رو فراخوانی میکردیم . پس اول یه توضیح کوتاه راجب کتابخانیش بدم بهتون ...

توضیح مختصر کتابخانه واژه ابر : این کتابخانه برای تبدیل داده های متنی انبوه به یک تصویر گرافیکی استفاده میشود . اینطوری که تعداد تکرار هر کلمه در متن ورودی را میگیرد و هر چقدر یک کلمه (در اینج مثلا ژانر ها یا هر ویژگی که کاربر خودش

بسمه تعالی

ورودی بده (در دیتابیس بیشتر تکرار شده باشد در تصویر نهایی با فونت بزرگتر بزرگتر و مرکزی تر نمایش داده میشود و کلماتی که تکرار کمتری دارند با فونت های کوچکتر و در حاشیه ها قرار میگیرند .

توی کدش هم که خیلی سادس اول میایم به متن از همه ویژگی های اهنگ ها (ویژگی ای که کاربر خودش ورودی داده) میسازیم بعد با استفاده از تابع آماده جنریت تو کتابخانه مون . تصویرمون رو میسازیم . بعدش نمودار ها خط ها درجه بندی ها را حذف میکنیم .

* برای جذاب تر شدن برنامه ام اومدم به عنوان به بخش امتیازی دیگه بهش موزیک بی کلام اضافه کردم . این کار رو با استفاده از کتابخانه پای گیم انجام دادم . (کتابخانه ای بسیار وسیع برای ساخت بازی و چیز های گرافیکی در پایتون و ...) . اینجوری که اول اومدم قبل از اولین حلقه وایل ترو تو تابع مین تو فایل مین وارد کتابخانه شدم بعد اهنگ بی کلامی که دانلود کرده و در فولدر پروژم بارگذاری کرده بودم را لود کردم بعد پخشش کردم (به این صورت که اگر زمانش تمام شد دوباره از اول پخش بشه اهنگم) بعد هم توی بخشی که کاربر میخواد به طور کامل از برنامه خارج بشه متوقفش کردم .

26 . دیگه نوبتی هم باشه نوبت میرسه به غول مراحل اخر . و تکمیل فایل نهایی یا همون فایل مین و اضافه و هندل کردن بخش های ترای اکسپت برای جاهای مورد نیاز پروژه برای جلوگیری از کرش کردن برنامه . اومدم تو فایل ابتدا همه موارد و مازول های لازم رو ایمپورت و فراخوانی کردم بعد داخل بدنه تابعی به اسم مین همه کد های لازم رو نوشتم و اینجوری شروع شده در اولین قدم به شی از کلاس دیتا لودر ساخته میشه بعد همه چیز از رو لیست اهنگ از اون شی و ... انجام می پذیره . تو منوی اصلی و اولیه با چندین گزینه (هندل کردن داده های گمشده و داده های پرت و بهره منده از توابع اماری نوشته شده در مازول مربوط به خودش و رسم انواع نمودار و اضافه کردن اهنگ جدید به فایل مون و ریست کردن دیتاست و سیو کردن تغییرات انجام شده .) بعد تو هر یک از بخش ها بازم منوی مربوط به خودشون رو گذاشتم . هر بخش برای خودش حلقه وایل ترو دارد و با هندل کردن این حلقه ها و همچنان برای تمامی بخش ها پرسش خروج از برنامه رو اوردم تا کاربر هر وقت خواست از برنامه خارج شه و برگرده عقب . بعدا برای هندل کردن برنامه مون و جلوگیری از ورودی های نامعتبر و نادرست و کرش شدن برنامه در تمامی بخش های لازم از ایف الس های مناسب و ترای اکسپت های مناسب استفاده کردیم

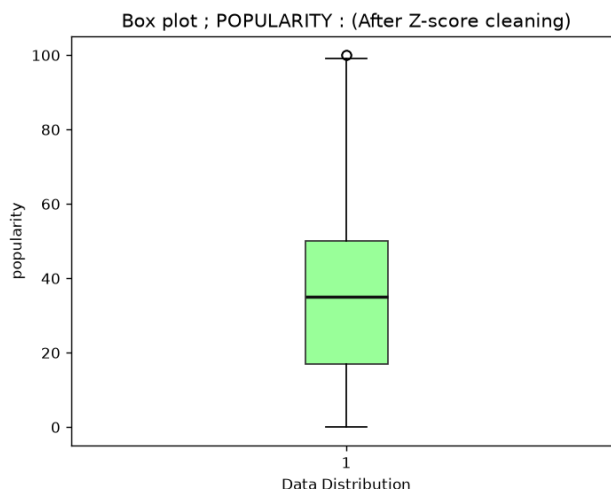
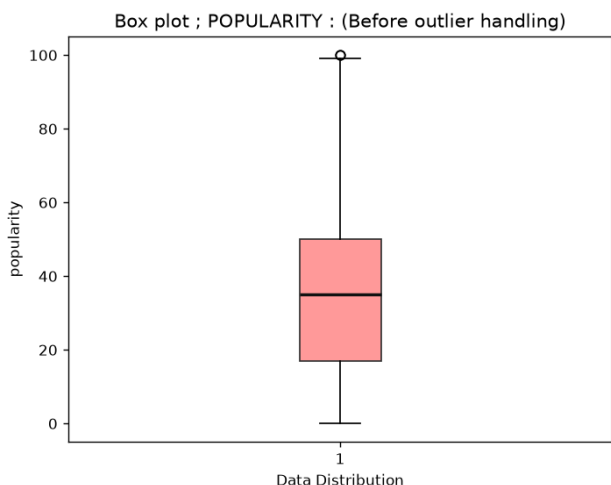
جمع بندی موارد امتیازی که پروژم پیاده سازی کردم :

1 . متد های جالب و مفید تو بخش کلاس انالیز دیتا . مثل : (دادن تاپ ترین اهنگ ها ، فیلتر دیتا براساس اسم هنرمند و ژانر . دادن اهنگ های شبیه اهنگ مورد علاقه کاربر) .

2 . اضافه کردن نمودار واژه ابر .

3 . اضافه اهنگ بی کلام برای پس زمینه برنامه .

الان تنها چیزی که موند اضافه کردن اسکرین شات از نمودار هاست که در جلو رو خواهیم آوردشان

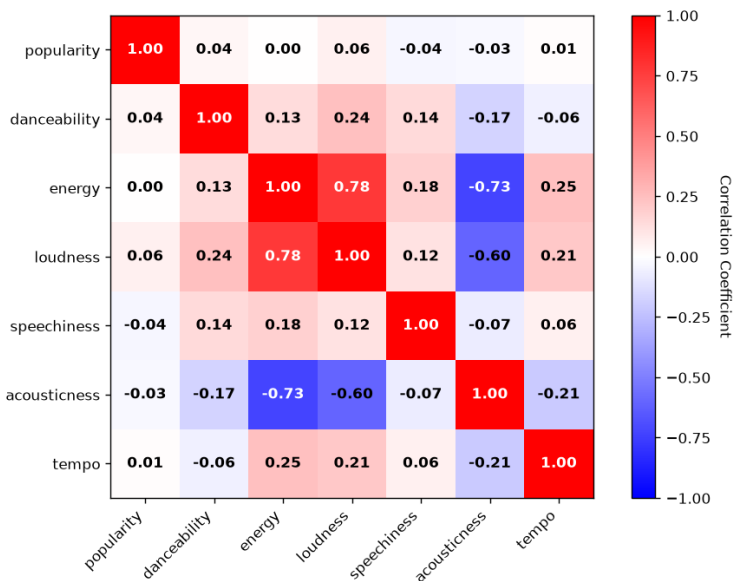


بِسْمِ تَعَالٰی

Wordcloud : track genre



Audio Features Correlation Matrix



بخوام یہ توضیح کوتاه راجب کتابخانه شمارنده (کانتر)

از کالکشن کتابخانه های استاندارد پایتون بدم :

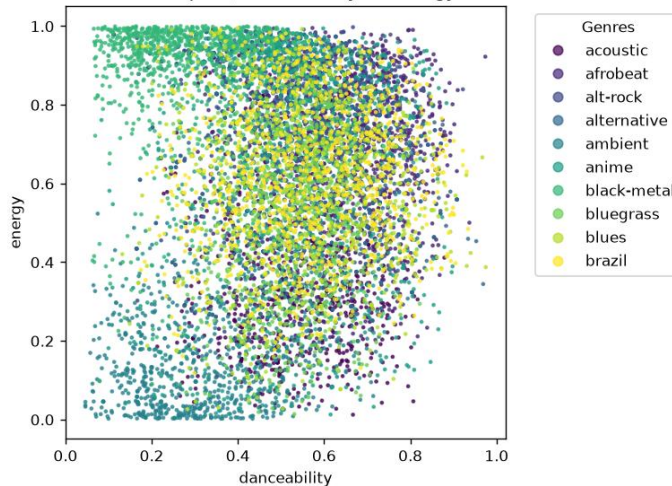
یک لیست قابل پیمایش مثلا لیست رو میگیره و تعداد

تکرار هر عضو رو حساب میکنه و خروجیش شبیه

دیکشنری هست که کلیدهای ان اعضای لیست و مقادیر

ان تعداد تکرار ان عضو هستند .

Scatter plot ; danceability vs energy



و اما تجربیات یادگیری: کاربرد واقعی مفاهیم شیءگرایی و جلوگیری از تکرار ساختارها

یکی از ارزشمندترین و جذاب ترین دستاوردهای این پروژه برای من ، تبدیل تفکر برنامه نویسی خطی به ساختار پیشرفته‌ی شیءگرایی برای مدیریت حجم بالای داده‌های واقعی بود. تجربیات این پروژه عبارتند از :

1. کپسوله‌سازی و اعتبارسنجی پویای داده‌ها .

در دنیای واقعی، داده‌های خام ممکن است مخدوش یا ناقص باشند. با طراحی **کلاس آهنگ**، هر سطر از جدول داده‌ها را به یک موجودیت مستقل نرم‌افزاری با هویت مشخص تبدیل کردم .

تجربه کاربردی: به جای رها کردن ویژگی‌ها به صورت متن یا عدد ساده، از ویژگی‌های ترتیبی و توابع و متد های تنظیم‌کننده (ابزار زبان پایتون مانند **پراپرتی ها** و ...) استفاده کردم. این ساختار مانند یک **دروازه امنیتی** عمل می‌کند؛ به طوری که موقع خواندن داده‌های مخدوش از فایل یا هنگام اضافه شدن آهنگ جدید توسط کاربر، ورودی‌ها بررسی شده و از ورود داده‌های **خارج از محدوده** **استاندارد** (مثلاً خارج از بازه صفر و یک برای شاخص‌های صوتی مثل انرژی و شادابی) جلوگیری می‌شود .

2. ارث‌بری و چندریختی .

در بخش تمیزکاری داده‌ها و **مدیریت داده‌های پرت**، با روش‌های مختلفی روبرو بودم (مانند پرت کردن با میانگین، میانه یا همسایگی)

تجربه کاربردی: با طراحی کلاس‌های پایه و تعریف رفتارهای عمومی، یک قرارداد ساختاری ایجاد کردم. سپس کلاس‌های فرزند این رفتارها را **ارث‌بری** کردند و وظایف خود را بازنویسی کردند. این کار باعث شد برنامه اصلی بدون درگیر شدن با جزئیات هر الگو، بتواند رفتار موردنظر را صدا بزند. همچنین اگر در **آینده** بخواهم الگوریتم جدیدی اضافه کنم، نیازی به تغییر کدهای قبلی ندارم.

3. تحقق اصل عدم تکرار در برنامه‌نویسی .

تکرار کدهای مشابه در پروژه‌های بزرگ، احتمال خطا را به شدت بالا می‌برد. در این پروژه با **دو** تکنیک از تکرار جلوگیری کردم:

تخصصی کردن وظایف : عملیات خواندن و نوشتن روی حافظه، محاسبات آماری و رسم نمودارها هر کدام به کلاس‌های مجزا واگذار کردم تا یک کد بارها در بخش‌های مختلف **تکرار نشود** .

حلقه‌های پویا در محاسبات: در بخش محاسبات همبستگی ویژگی‌های صوتی، به جای نوشتن فرمول‌های تکراری برای تک‌تک جفت‌ویژگی‌ها، از **حلقه‌های تو در تو** روی لیست ویژگی‌ها استفاده کردم. این کار باعث شد **محاسبات تمام شاخص‌ها** تنها در چند خط کوتاه و پویا انجام شود .

خب به عنوان حرف های آخر تو روزای آخر مهلت پروژه اومدم تو فایل دیتای اولیه تغییرات کوچیکی ایجاد کردم . به ستون ویژگی عددی با عنوان شماره آهنگ ایجاد کردم و محتویات داخل ستون " محتوای بی پرده " رو از ترو فالز به صفر و یک تو فایل نهایی تغییر دادم . و اومدم از ویژگی پراپرتی و گتر و ستر پایتون برای تمام ویژگی های عددی مان استفاده کردم تا شناسایی داده های گمشده راحت تر شود . (بازه های معتبر هر ویژگی را از سایت مربوطه استخراج کردم) . و اومدم چند تا شرط دیگه برای ورودی های کاربر گذاشتم تا برنامه در هیچ صورتی کرش نکند (تمایز بین ویژگی های عددی و رشته ای) . و تغییراتی هم در نحوه چاپ اطلاعات تو خروجی دادم .

لینک گیت هاب پروژه : [پروژه برنامه نویسی پیشرفته پایتون سامانه موسیقی](#)

چیزی که به عنوان بخش آخر به پروژه اضافه کردم : بخش گزارش کارهایی که (هندل کردن داده های پرت و گمشده) انجام داده بودیم . و لیست شماره داده هایی که دارای رشته گمشده تو یکی از ویژگی هاشون بودن (که اون آهنگ رو حذف کردیم از لیست).